Universidad Nacional de Las Ciencias
Ingeniería en Ciberseguridad
Programación
Alumnos:
Nelson Sánchez, Cédula: 19.264.908
Andrea Pernil, Cédula: 32.327.658.
Pedro Diaz, Cédula: 31.887.476.
Sección: S18

Taller #5 Módulo 4 "Realice un escaneo de seguridad OWASP a sistemas transaccionales, validar que se aplican en su proyecto"

Proyecto #3 (Monitor de Integridad de Archivos - FIM)

1. INTRODUCCIÓN Y MARCO DE EVALUACIÓN OWASP TOP 10

El estándar **OWASP Top 10** define las diez principales amenazas y riesgos de seguridad en aplicaciones de software y sistemas transaccionales. El objetivo de este taller es realizar una auditoría de seguridad sobre el software **FIM-Secure**, identificar posibles vectores de vulnerabilidad en la capa de datos e interacción con el sistema de archivos, y presentar las correspondientes estrategias de remediación y codificación defensiva.

2. MATRIZ DE EVALUACIÓN DE VULNERABILIDADES OWASP EN FIM-SECURE

Categoría OWASP	Riesgo Identificado en FIM	Impacto Potencial	Estado de Remediación
A03:2021 Inyección (SQLi)	Manipulación de consultas SQL al consultar o registrar rutas de archivos en la BD.	**Alto:** Burlado de auditoría, lectura desautorizada o alteración de registros.	**REMEDIADO** (Consultas Parametrizadas)

A01:2021 Broken Access Control	Acceso o modificación no autorizada de la base de datos de baseline (fim_secure.db).	**Crítico:** Alteración maliciosa de los hashes base por un atacante local.	**REMEDIADO** (Permisos ACLs en SO)
A08:2021 Software & Data Integrity Failures	Falsificación o adulteración del binario en C (fim_hasher.exe).	**Crítico:** Reemplazo del motor por un ejecutable troyanizado.	**REMEDIADO** (Verificación de Firma)

3. CASO 1: REMEDIACIÓN DE A03:2021 - INYECCIÓN SQL (SQLI)

Ejemplo práctico de vulnerabilidad detectada en la búsqueda transaccional y su código corregido:

VULNERABLE: Concatenación Directa de Parámetros

```
# CÓDIGO VULNERABLE DETECTADO EN EVALUACIÓN OWASP
def consultar_alerta_insegura(nombre_archivo):
    # La concatenacion directa permite inyectar comandos SQL
    sql = "SELECT * FROM ARCHIVO WHERE ruta_absoluta LIKE '%" +
nombre_archivo + "%';"
    cursor.execute(sql) # VULNERABLE A SQL INJECTION
```

REMEDIACIÓN: Uso de Sentencias Preparadas

```
# CÓDIGO REMEDIADO E IMPLEMENTADO EN FIM-SECURE
def consultar_alerta_segura(nombre_archivo):
    # La variable se aisla utilizando marcadores de posición '?'
    sql = "SELECT * FROM ARCHIVO WHERE ruta_absoluta LIKE ?;"
    parametro = f"%{nombre_archivo}%"
    cursor.execute(sql, (parametro,)) # 100% SEGURO Y PARAMETRIZADO
```

4. CASO 2: REMEDIACIÓN DE A01:2021 - CONTROL DE ACCESO ROTO

Si un atacante sin privilegios modifica directamente el archivo de base de datos local (fim_secure.db), la integridad de todo el sistema se compromete. Para remediar este riesgo, el orquestador en Python verifica y aplica permisos restringidos en el sistema operativo (NTFS ACLs en Windows):

```
import os, sys, win32security, ntsecuritycon

def asegurar_permisos_bd(db_path):
    """Restringe acceso a la BD SQLite permitiendo únicamente acceso al usuario Administrador/SYSTEM"""
    if os.path.exists(db_path):
        # Obtiene el Descriptor de Seguridad del Archivo
        sd = win32security.GetFileSecurity(db_path, win32security.DACL_SECURITY_INFORMATION)
        dacl = win32security.ACL()

        # Otorga Control Total solo a Administradores locales
        admins_sid, _, _ = win32security.LookupAccountName("", "Administradores")
        dacl.AddAccessAllowedAce(win32security.ACL_REVISION, ntsecuritycon.FILE_ALL_ACCESS, admins_sid)

        # Aplica la nueva DACL al archivo de base de datos
        sd.SetSecurityDescriptorDacl(1, dacl, 0)
        win32security.SetFileSecurity(db_path, win32security.DACL_SECURITY_INFORMATION, sd)
        print("[REMEDIACIÓN OWASP A01] Permisos de base de datos restringidos a Administradores.")
```

5. CASO 3: REMEDIACIÓN DE A08:2021 - FALLOS DE INTEGRIDAD EN EL BINARIO C

Antes de invocar la ejecución del ejecutable fim_hasher.exe mediante subprocess, el orquestador valida el hash SHA-256 del propio binario para garantizar que no haya sido alterado por un malware:

```
import hashlib, os

HASH_BINARIO_OFICIAL = "9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a08"
```

```
def validar_integridad_ejecutable_c(path_exe):
    """Verifica que el ejecutable en C no haya sido modificado antes de su uso"""
    if not os.path.exists(path_exe):
        raise FileNotFoundError("Binario C no encontrado.")

    hasher = hashlib.sha256()
    with open(path_exe, 'rb') as f:
        while chunk := f.read(4096):
            hasher.update(chunk)

    if hasher.hexdigest() != HASH_BINARIO_OFICIAL:
        raise SecurityError("[ALERTA CRÍTICA OWASP A08] El binario fim_hasher.exe
ha sido adulterado.")

    print("[REMEDIACIÓN OWASP A08] Integridad del binario C verificada
correctamente.")
```

6. CONCLUSIONES DEL TALLER N° 5

La evaluación de vulnerabilidades basada en el estándar OWASP Top 10 permitió identificar y mitigar proactivamente los riesgos transaccionales y de sistema en **FIM-Secure**. Las soluciones implementadas (parametrización SQL, ACLs en el sistema de archivos y verificación de la firma del binario) garantizan la máxima robustez defensiva exigida en entornos profesionales de ciberseguridad.